阿里云 × CLOUD NATIVE COMPUTING FOUNDATION

云原生技术公开课

第 21 讲

Kubernetes 存储架构及插件使用

阚俊宝 阿里巴巴技术专家

1 Kubernetes 存储体系架构

2 Flexvolume 介绍及使用

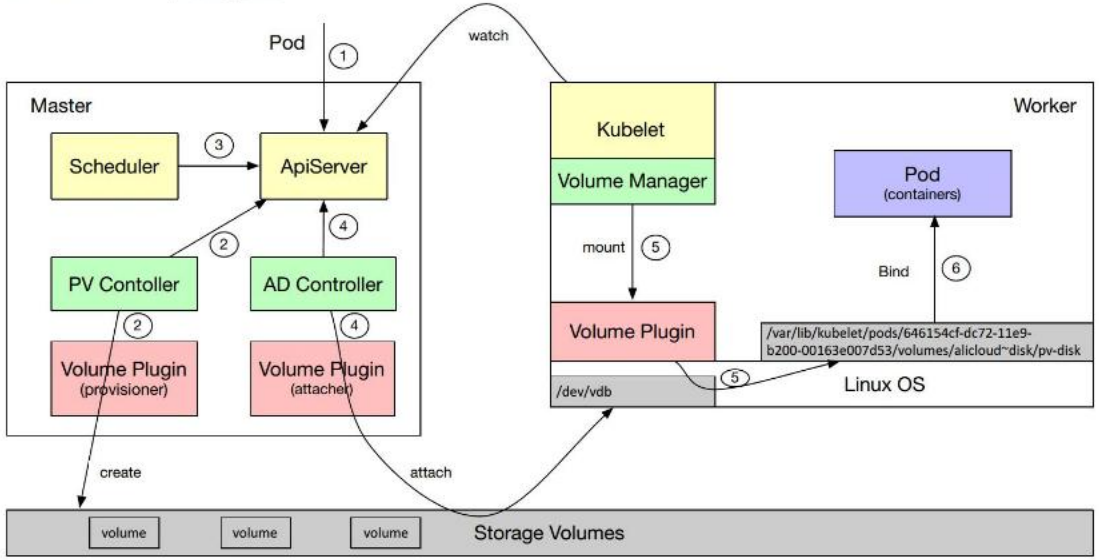
3 CSI 介绍及使用

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

Kubernetes挂载Volume过程

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pvc-disk
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: alicloud-disk-ssd
  resources:
    requests:
      storage: 20Gi

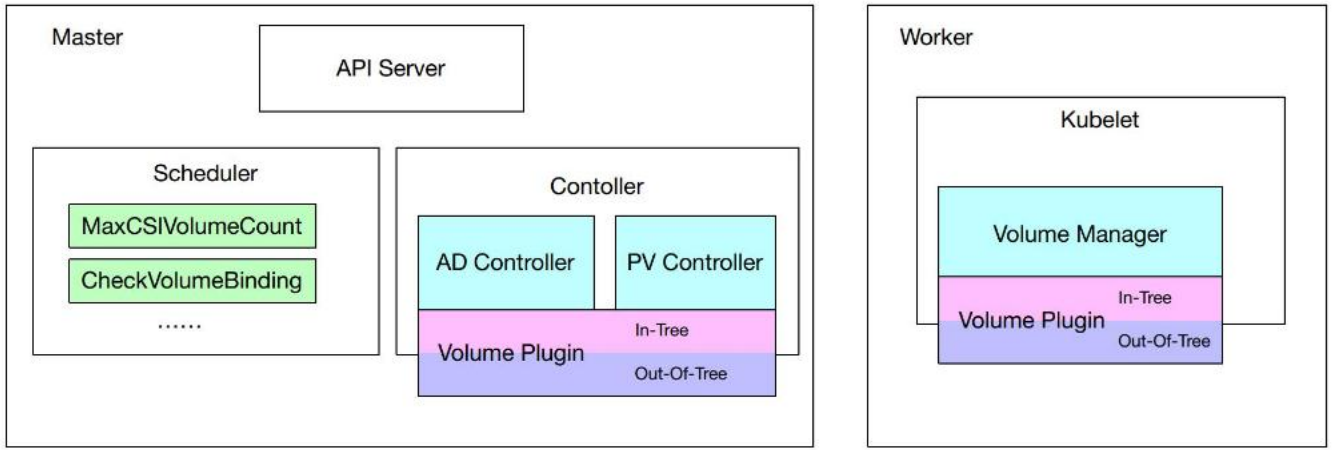
---
apiVersion: apps/v1beta2
kind: StatefulSet
metadata:
  name: web
spec:
  selector:
    matchLabels:
      app: nginx
  serviceName: "nginx"
  replicas: 1
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
          name: web
        volumeMounts:
        - name: disk-pvc
          mountPath: /data
```



1. 用户创建一个包含PVC的Pod（使用动态存储卷）；
2. PV Controller发现这个PVC处于待绑定状态，调用Volume Plugin(in-tree或out-of-tree)创建存储卷，并创建PV对象；并将创建的PV与PVC绑定；
3. Scheduler根据Pod配置、节点状态、PV配置等信息，把pod调度到worker节点Node上；
4. AD Controller发现Pod和PVC处于待挂载状态，调用Volume Plugin(in-tree或out-of-tree)实现设备挂载到目标节点(/dev/vdb)；
5. 在Worker节点上，Kubelet(Volume Manager)等待设备挂载完成，通过Volume Plugin将设备挂载到指定目录: /var/lib/kubelet/pods/646154cf-dc72-11e9-b200-00163e007d53/volumes/alicloud~disk/pv-disk
6. Kubelet在被告知挂载目录准备好后，启动Pod中的containers，用Docker -v方式(bind)将已经挂载到本地的卷映射到容器中；

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

Kubernetes 存储架构



Provision ➔ Attach ➔ Mount; Unmount ➔ Detach ➔ Delete;

- PV Controller：负责PV/PVC的绑定、生命周期管理，并根据需求进行数据卷的Provision/Delete操作；
- AD Controller：负责存储设备的Attach/Detach操作，将设备挂载到目标节点；
- Volume Manager：管理卷的Mount/Unmount操作、卷设备的格式化等；
- Volume Plugins：扩展各种存储类型的卷管理能力，实现第三方存储的各种操作能力与Kubernetes存储系统结合；
- Scheduler: 实现Pod调度能力，存储相关的调度器实现了针对存储卷配置进行调度；

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

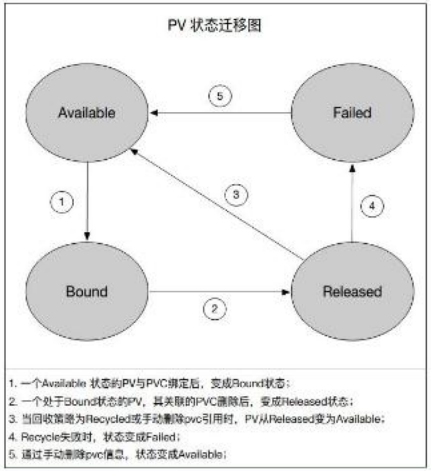
PV Controller介绍

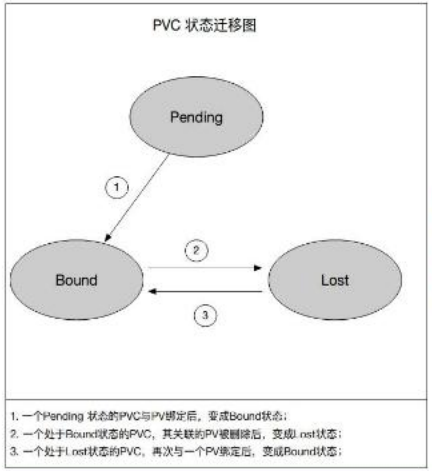
主要概念：

- Persistent Volume: 持久化存储卷，详细定义预挂载存储空间的各项参数；无NameSpace限制，一般由Admin创建维护PV；
- Persistent Volume Claim：持久化存储卷声明，用户使用的存储接口，对存储细节无感知，属于某NameSpace内；
- StorageClass：存储类，创建PV存储的模板类，即系统会按照StorageClass定义的存储模板创建存储卷（包括真实存储空间和PV对象）；

主要任务：

- PV、PVC生命周期管理：创建、删除PV对象；负责pv、pvc的状态迁移；
- 绑定PVC、PV对象：一个PVC必须与一个PV绑定后才能被应用使用，pv-controller会根据绑定条件和对象状态对PV、PVC进行Bound、Unbound操作；



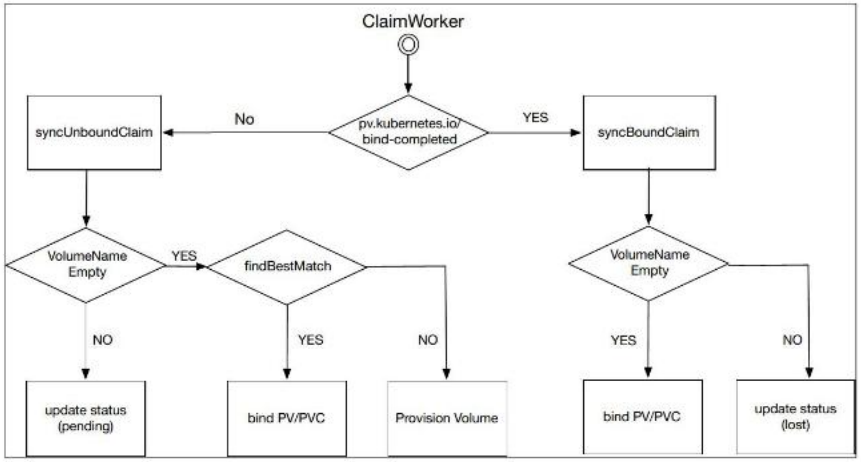


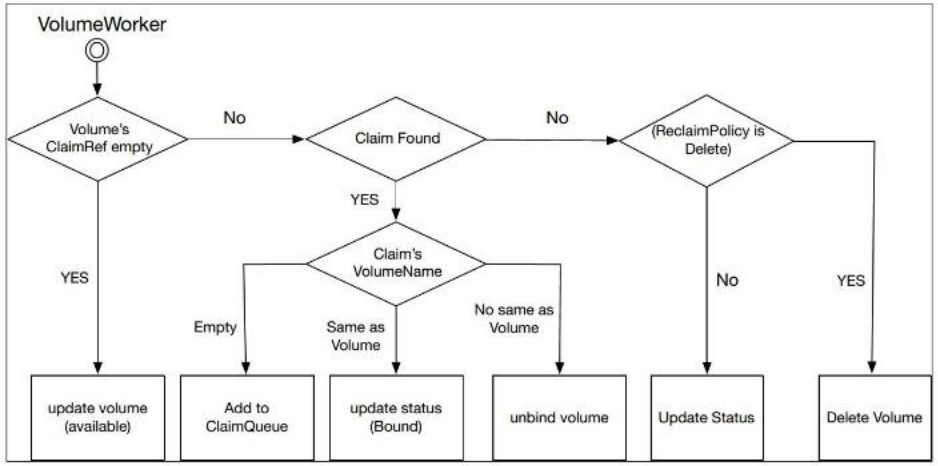
绑定条件

VolumeMode 检查	筛选所有具有相同VolumeMode的volume；Block/FileSystem；
LabelSelector 检查	若PVC配置了selector,matchLabels，会筛选符合Selector条件的Volume；
StorageClassName 检查	如果配置了StorageClassName，会筛选value值相同的Volume；
AccessMode 检查	筛选PVC的AccessMode列表中所有选项，都有相应的AccessMode配置的volume；
Size 检查	筛选哪些大于等于PVC的Size 的volume，若数量大于1，则筛选size最小的volume；

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

PV Controller 实现





ClaimWorker:

- 实现PVC的状态迁移;
- 通过系统标签”pv.kubernetes.io/bind-completed”来标识一个pvc是为Bound状态;
- 当pvc为Unbound时，触发PV筛选逻辑，如果找到合适的PV则bound，若果找不到则触发Provision（若不是In-Tree Provisioner则等待）;

VolumeWorker:

- 实现PV的状态迁移;
- 通过ClaimRef来判断PV是Bound还是Released；PV状态为Released时，若Reclaim为Delete，则执行删除逻辑;

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

AD Controller介绍

AD Controller负责将数据卷挂载/卸载到特定节点上；

核心对象：DesiredStateOfWolrd, ActualStateOfWorld；
核心逻辑：Reconcile, desiredStateOfWorldPopulator；

DesiredStateOfWolrd：集群中预期要达到的数据卷挂载状态；
ActualStateOfWorld：集群中实际存在的数据卷挂载状态；

desiredStateOfWorldPopulator：根据卷挂载状态，更新DSW、ASW数据；
reconcile：根据DSW、ASW对象状态，轮询执行Attach、Detach操作；

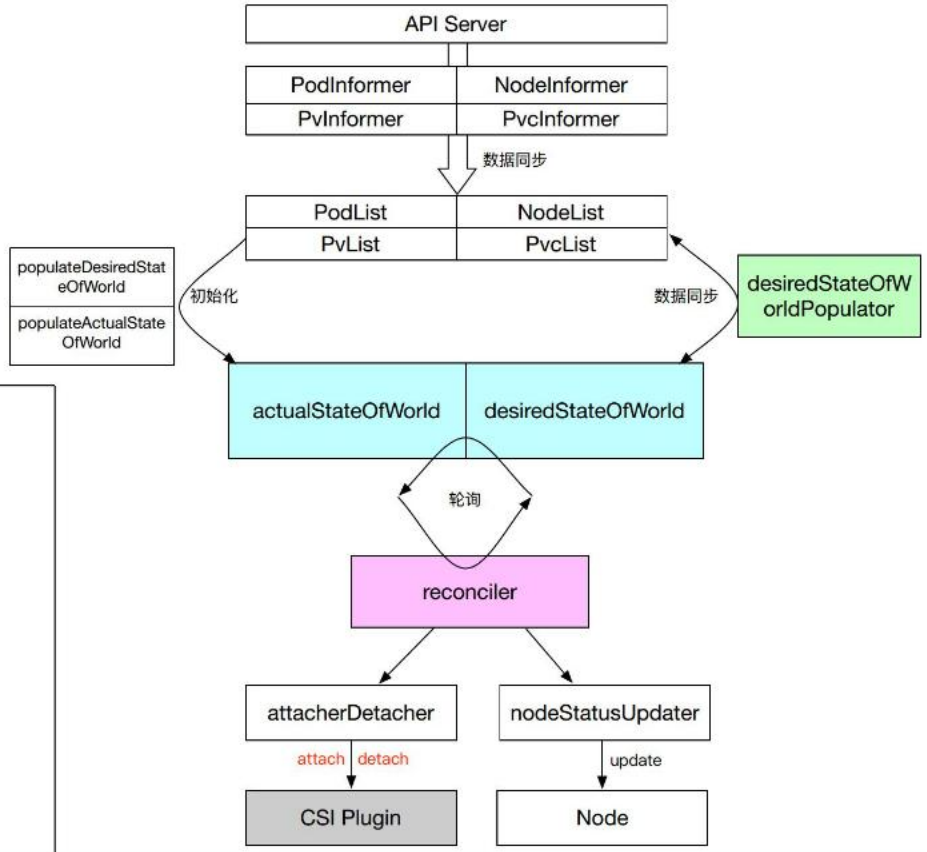
```
//desiredStateOfWorld
{
   "nodesManaged":{
      "worker1": {
         "nodeName":"worker1",
         "volumesToAttach":[
            {
               "volName": "volume1",
               "scheduledPods":[],
            },
            {
               "volName":"volume2",
               "scheduledPods":[],
            },
         ]
      },

      "worker2": {
         "nodeName":"worker2",
         "volumesToAttach":[
         ]
      }
   },

   "volumePluginMgr":{"plugins": {}, "probe":""}
}
```

```
//actualStateOfWorld
{
   "attachVolumes":{
      "volume1": {
         "volumeName":"volume1",
         "nodeAttachedTo":[
            {
               "nodeName":"worker1",
               "mountByNode":true,
            }
         ]
      },
      "volume2": {
         "volumeName":"volume2",
         "nodeAttachedTo":[
            {
               "nodeName":"worker1",
               "mountByNode":true,
            }
         ]
      },
   },

   "volumePluginMgr":{"plugins": {}, "probe":""}
}
```



阿里云 CLOUD NATIVE COMPUTING FOUNDATION

Volume Manager 介绍

- 是Kubelet中的一个Manager，调用本节点Volume的Attach/Detach/Mount/Unmount操作；
- volumeManager实时扫描本节点的Pod状态，对需要更新状态的Volume通过调用volume Plugin执行执行相应操作；

数据结构：

- volumePluginMgr：管理本节点上In-Tree/Out-Of-Tree插件列表；
- desiredStateOfWorld: 记录节点上数据卷的期望挂载状态；
- actualStateOfWorld: 记录节点上数据卷的实际挂载状态；

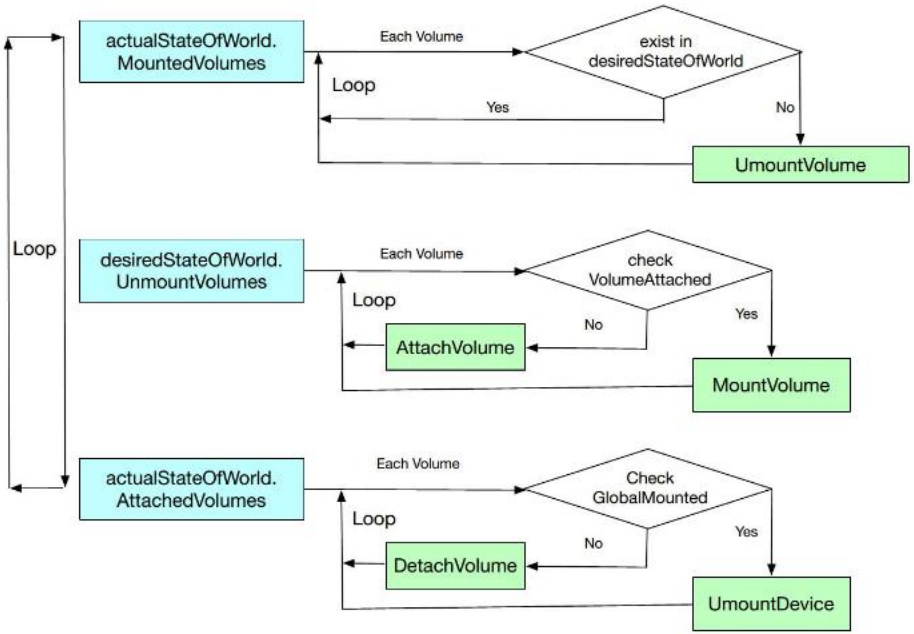
核心逻辑：

- desiredStateOfWorldPopulator：同步包含数据卷的pod状态；
- Reconciler：循环执行Attach、Detach、Mount、Unmount，具体操作通过调用volume plugin实现的接口完成；

AD Controller 还是 Volume Manager做Attach/Detch操作?

- 通过--enable-controller-attach-detach来控制；

VolumeManager根据存储类型需要还会执行一些公共操作，例如：块设备的格式化、挂载块设备到公共目录等。



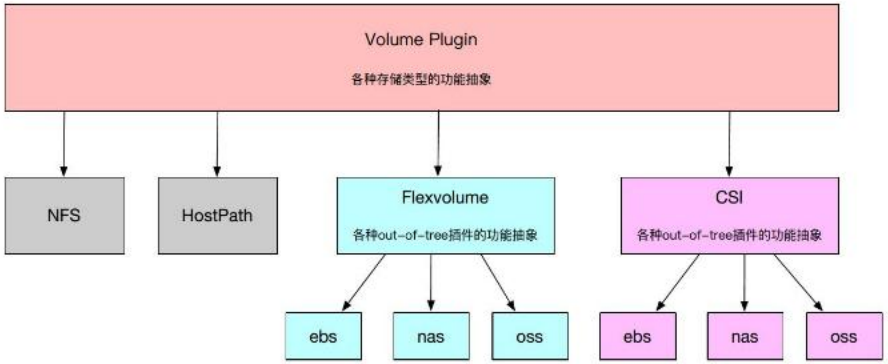
阿里云 CLOUD NATIVE COMPUTING FOUNDATION

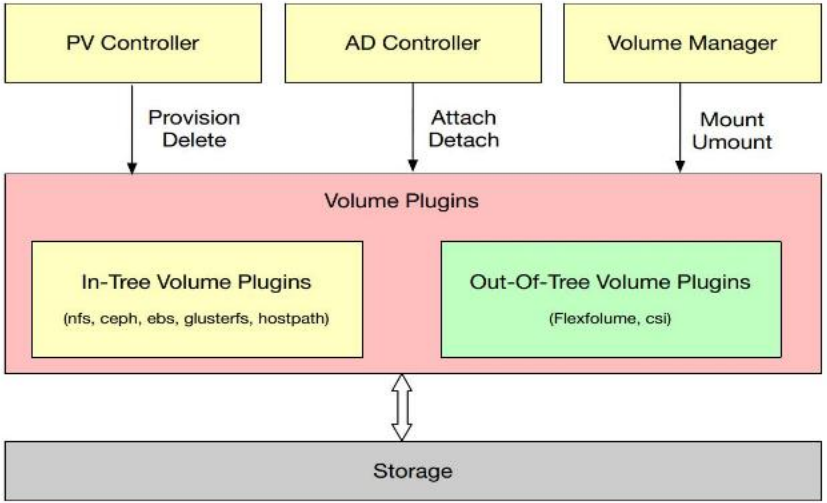
Volume Plugins介绍

根据不同的存储类型提供数据卷Provision、Delete、Attach、Detach、Mount、Unmount具体操作实现，是多种具体存储类型的功能抽象接口。

根据源码的位置可分为In-Tree和Out-Of-Tree两类：

- In-Tree：在Kubernetes源码内部实现，和Kubernetes一起发布、管理，更新迭代慢、灵活性差；
- Out-Of-Tree: 代码独立于Kubernetes，由存储尝试实现；CSI/Flexvolume是两种机制，可以根据存储类型实现为不同的存储插件；





In-Tree		Out-Of-Tree
• cephfs • cinder • configMap • downwardAPI • emptyDir • fc (fibre channel) • Flocker • glusterfs • hostPath • Iscsi • local	• nfs • persistentVolumeClaim • projected • portworxVolume • quobyte • rbd • scaleIO • secret • Storageos	• CSI • FlexVolume

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

Volume Plugins 插件管理

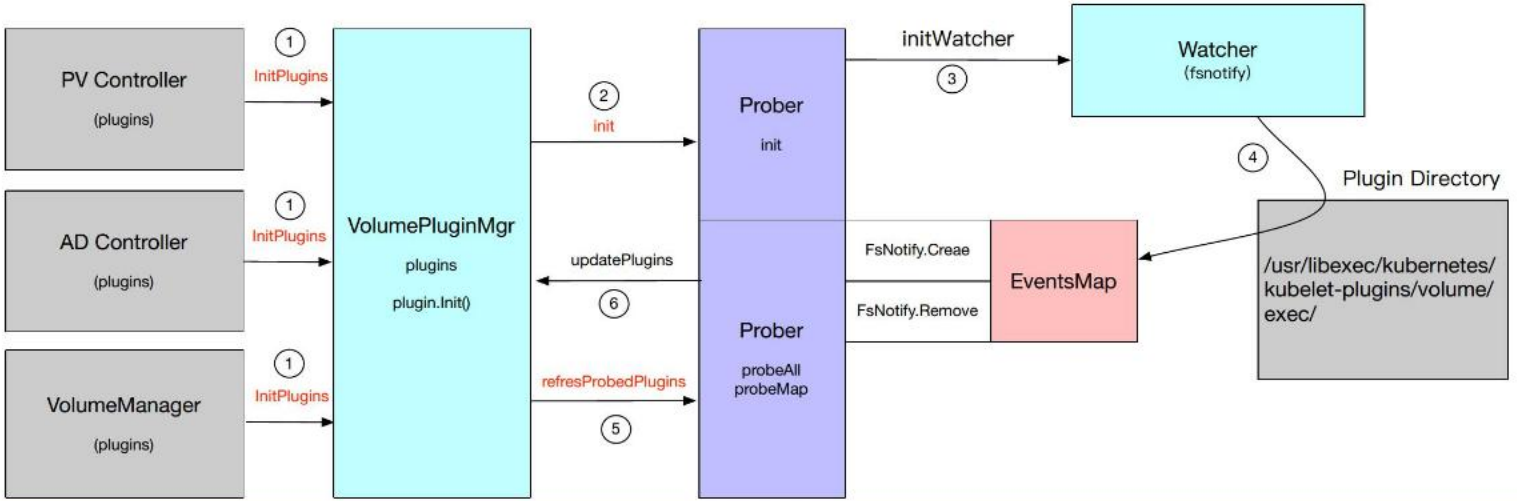
VolumePluginMgr：管理存储插件，Kubelet、PV Controller、AD Controller初始化时创建，负责插件列表的维护（添加/删除）；

可管理Kubernetes原生自带的插件和动态添加、删除的插件；

数据结构：
- Plugins: VolumePluginMgr管理的插件列表，随着插件的变化而更新；
- Prober：探针，用来发现新的Plugin，实现插件的添加、删除；

运行时：
- InitPlugins：初始化各个Plugin，并初始化Probe，使其watcher目标目录；
- refreshProbedPlugins：通过探针收集的Events，刷新当前Plugin列表；

Probe探针：
- 每次刷新Plugin返回ProbeEvent，Event类型包括：ProbeAddOrUpdate/ProbeRemove；
- ProbeAddOrUpdate事件触发添加、初始化新Plugin；
- ProbeRemove事件触发从Plugins中删除插件；



Kuberenetes 存储卷调度

- Kubernetes在调度Pod的时候会通过不同调度器对节点进行筛选，其中部分调度器是Volume相关的调度器；
- VolumeZonePredicate：会检查PV的Label标签，如果PV的Label中存在failure-domain.beta.kubernetes.io/zone标签，则会对Node进行筛选，只有Node的Label中有failure-domain.beta.kubernetes.io/zone标签的，且值相同的node才被选中；
- VolumeBindingPredicate：检查PV的nodeAffinity配置，使用matchExpressions定义的参数与Node的Label进行匹配对比，符合条件的node会被选中；
- CSIMaxVolumeLimitPredicate：节点的某个插件类型已经挂载的卷个数小于限额时才会选中；

VolumeZonePredicate	检查Pod volume的zone标签和节点的zone标签是否匹配； failure-domain.beta.kubernetes.io/zone
NoDiskConflict	检查AWS/GCE/RBD，是否已经挂载到节点上了；
CSIMaxVolumeLimitPredicate	检查CSI类型的volume在某个节点的最大挂载数量；
VolumeBindingPredicate	检查Pod对应的Local Volume的nodeAffinity和节点上的标签是否对应；

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-disk
  labels:
    failure-domain.beta.kubernetes.io/zone: cn-shenzhen-a
    failure-domain.beta.kubernetes.io/region: cn-shenzhen
spec:
  capacity:
    storage: 20Gi
  accessModes:
    - ReadWriteOnce
  flexVolume:
    driver: "alicloud/disk"
    fsType: "ext4"
    options:
      volumeId: "d-wz9bkj7jhbb4jo77n1tz"
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: example-pv
spec:
  capacity:
    storage: 100Gi
  accessModes:
  - ReadWriteOnce
  storageClassName: local-storage
  local:
    path: /mnt/disks/ssd
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - key: kubernetes.io/hostname
          operator: In
          values:
          - example-node
```


阿里云 CLOUD NATIVE COMPUTING FOUNDATION

1

Kubernetes
存储体系架构

2

Flexvolume
介绍及使用

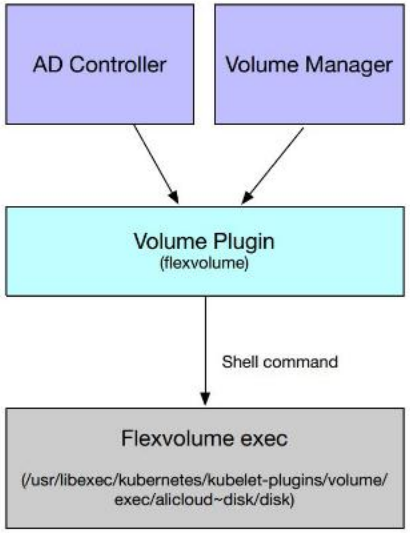
3

CSI
介绍及使用

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

Flexvolume 介绍

- Flexvolume实现了VolumePlugin的Attach/Detach/Mount/Unmount接口定义，这些接口由第三方实现，并按照一定规则调用接口。
- Flexvolume是一个由Kubelet驱动的可执行文件，每一次接口调用均为一次可执行文件的命令行调用；不是常驻内存的守护进程；
- 调用Flexvolume的Stdout作为kubelet调用的返回结果，要求JSON格式；
- 默认存放地址：/usr/libexec/kubernetes/kubelet-plugins/volume/exec/alicloud~disk/disk
- 所有挂载均先将远程设备挂载到本地主机，然后从主机通过bind映射到容器内部；



调用命令格式：

{flexvolume-binary} {action} {parameters}

调用示例：

/usr/libexec/kubernetes/kubelet-plugins/volume/exec/alicloud~disk/disk attach
{ “VolumeId”: “d-8vb4fflsonz21h31cmss”, “kubernetes.io/fsType”: “ext4”, “kubernetes.io/pvOrVolumeName”: “d-8vb4fflsonz21h31cmss”, “kubernetes.io/readwrite”: “rw” }
cn-hangzhou.192.168.1.41

返回成功：

{ “status”: “Success”, “device”: “/dev/vdb”, "volumeName":""}

返回失败：

{ “status”: “Failure”, “message”: ” error message”, “volumeName”: “” }

返回结果格式：

```
type DriverStatus struct {
    Status string `json:"status"`
    Message string `json:"message,omitempty"`
    DevicePath string `json:"device,omitempty"`
    VolumeName string `json:"volumeName,omitempty"`
    Attached bool `json:"attached,omitempty"`
    Capabilities *DriverCapabilities `json:",omitempty"`
    ActualVolumeSize int64
`json:"volumeNewSize,omitempty"`
}

type DriverCapabilities struct {
    Attach          bool `json:"attach"`
    SELinuxRelabel  bool `json:"selinuxRelabel"`
    SupportsMetrics bool `json:"supportsMetrics"`
    FSGroup         bool `json:"fsGroup"`
    RequiresFSResize bool `json:"requiresFSResize"`
}
```


阿里云 CLOUD NATIVE COMPUTING FOUNDATION

Flexvolume 接口介绍

Init	初始化操作，部署插件、更新插件的时候执行初始化；
GetVolumeName	获取volume名，都使用默认实现返回原来的volumeName；
Attach	挂载存储盘；--enable-controller-attach-detach控制由AD Controller或kubelet发起；
WaitforAttach	Attach经常是异步执行，需要等待挂载完成，并获取挂载设备名；
MountDevice	将磁盘设备在本地挂载到GlobalMount目录，供本地各个Pod分别挂载使用；
GetPath	获取每个Pod对应的本地挂载目录，此目录作为容器启动时bind的主机目录；
SetUp	从GlobalPath目录挂载到Pod本地目录，使用Bind方式；
TearDown	Unmount Pod本地目录，并删除；
UnmountDevice	Unmount GlobalPath本地目录，并删除；
Detach	卸载磁盘，同样--enable-controller-attach-detach控制由AD Controller或kubelet发起；
ExpandVolumeDevice	扩容存储卷，由expand controller发起调用，需要在master节点执行(Expand Controller)；
NodeExpand	扩容文件系统，有kubelet(VolumeManager)发起调用，需要在挂载卷所在节点执行；

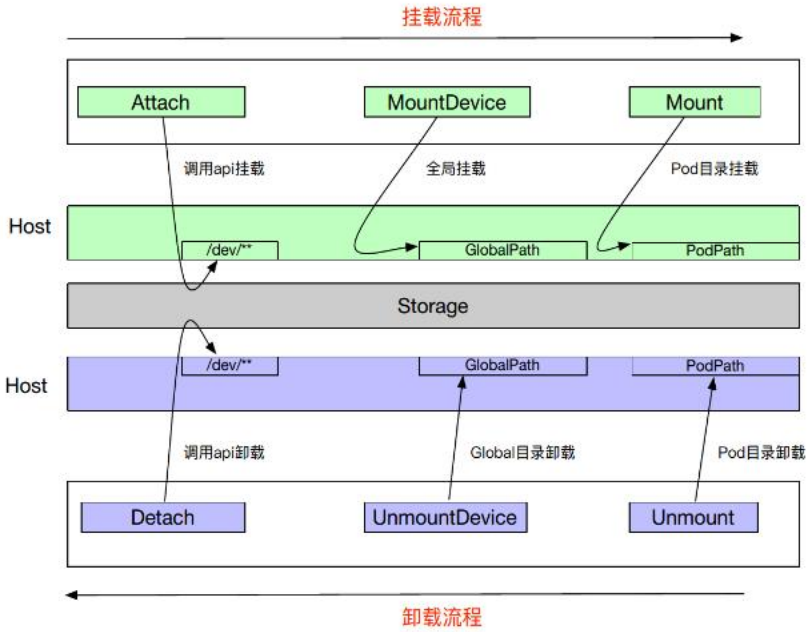
- Init 接口通过DriverCapabilities信息告诉Kubelet插件是否支持挂载、监控数据采集、文件系统扩容等信息；
- Flexvolume不需要对所有接口提供支持，不支持的接口返回NotSupport。这时VolumePlugin的In-Tree部分Flexvolume默认实现会被调用。

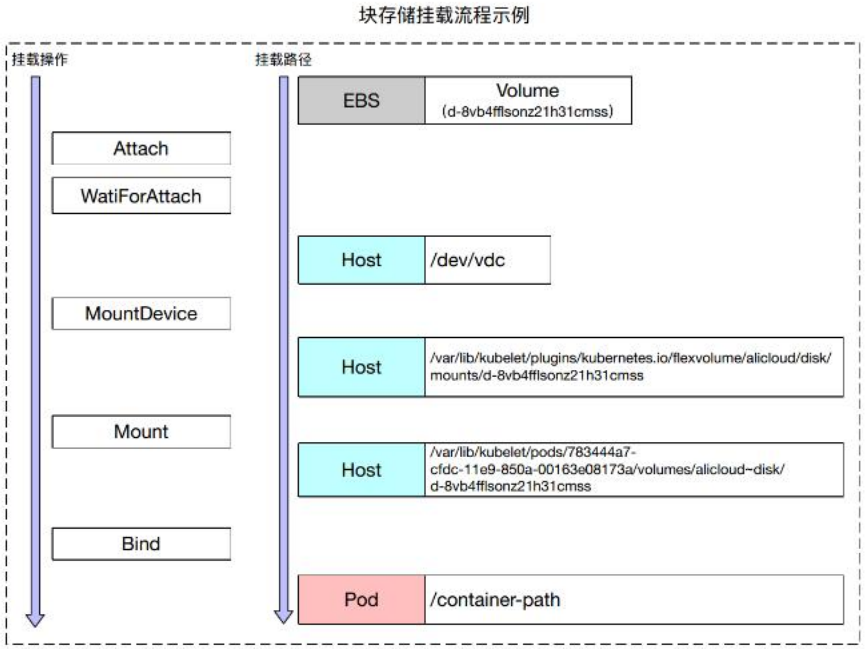
```
{
    “status”：“Not supported”，
    “message”：”error message”
}
```

- 对应需要使用用户名密码信息的接口，可以通过Secret方式输入：

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-oss
spec:
  flexVolume:
    driver: "alicloud/oss"
    secretRef:
      name: "osssecret"
    options:
      bucket: ""
```

Flexvolume 挂载分析





- 块存储需要Attach、格式化、Mount等操作，所以流程较长，步骤多；
- MountDevice是Mount的一部分，执行一些公共操作：格式化、挂载device到GlobalPath等；
- 文件存储无需Attach操作，只需从远端mount到主机，流程要简单；
- 开发Flexvolume驱动的过程即为实现Flexvolume接口的过程，实现中通过调用OpenApi等方式挂载第三方存储设备；

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

Flexvolume 代码示例

```
### Example: save script at /usr/libexec/kubernetes/kubelet-plugins/volume/exec/alicloud~cifs/cifs

init() {
  assertBinaryInstalled mount.cifs cifs-utils
  assertBinaryInstalled jq jq
  echo '{ "status": "Success", "message": "The fstab/cifs flexvolume plugin was initialized successfully",
"capabilities": { "attach": false } }'
  exit 0
}

assertBinaryInstalled() {
  binary="$1"
  package="$2"
}

errorExit() {
  if [[ $# -ne 1 ]] ; then
    echo '{ "status": "Failure", "message": "Unknown error in the fstab/cifs flexvolume plugin." }'
  else
    jq -Mcn --arg message "$1" '{ "status": "Failure", "message": $message }'
  fi
  exit 1
}

doMount() {
  if [[ -z ${1:-} || -z ${2:-} ]] ; then
    errorExit "cifs mount: syntax error. usage: cifs mount <mount dir> <json options>"
  fi
  mountPoint="$1"

******

  export PASSWD="$cifsPassword"
  result=$(mount -t cifs "$networkPath" "$mountPoint" -o "username=$cifsUsername,$mountOptions"
2>&1)
  if [[ $? -ne 0 ]] ; then
    errorExit "cifs mount: failed to mount the network path: $result"
  fi
  echo '{ "status": "Success" }'
  exit 0
}
```

```
doUnmount() {
  if [[ -z ${1:-} ]] ; then
    errorExit "cifs unmount: syntax error. usage: cifs unmount <mount dir>"
  fi
  mountPoint="$1"
  if [[ $(mountpoint "$mountPoint") != **is a mountpoint** ]] ; then
    errorExit "cifs unmount: no filesystem mounted under directory: '$mountPoint'"
  fi
  result=$(umount "$mountPoint" 2>&1)
  if [[ $? -ne 0 ]] ; then
    errorExit "cifs unmount: failed to unmount the network path: $result"
  fi
  echo '{ "status": "Success" }'
  exit 0
}

not_supported() {
  echo '{ "status": "Not supported" }'
  exit 1
}

command=${1:-}
if [[ -n $command ]]; then
  shift
fi

case "$command" in
  init)
    init "$@"
    ;;
  mount)
    doMount "$@"
    ;;
  unmount)
    doUnmount "$@"
    ;;
  *)
    not_supported "$@"
    ;;
esac
```

/usr/libexec/kubernetes/kubelet-plugins/volume/exec/alicloud~cifs/cifs mount /var/lib/kubelet/plugins/kubernetes.io/flexvolume/alicloud/cifs/mounts/pv-cifs “{********}”

项目示例：https://github.com/AliyunContainerService/flexvolume

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

Flexvolume 使用示例

Flexvolume存储卷的定义格式和其他In-Tree存储类型格式一致，区别在存储类型为Flexvolume；

Flexvolume参数定义：

- driver：定义插件类型，会根据这个参数值到某个目录下面找插件的可执行文件；
- fsType：定义存储卷文件系统类型；
- options：为一个Map参数类型，可以定义所有与存储类型、挂载相关的具体参数；

可以通过Selector、Label的方式，定向配置PV、PVC绑定关系；PV可以通过Label或者NodeAffinity配置调度信息；

Flexvolume存储卷可以使用静态存储或者动态存储方式，动态存储卷需要在集群中部署Provisioner插件；

右面例子可以看出：

- EBS存储卷（ID d-wz9bkj7jhbb4jo77n1tz）先挂载到主机上，设备名为/dev/vdb设备名；
- 然后/dev/vdb挂载到一个全局目录；
- 然后再挂载到Pod所在目录；
- 最后从Pod目录映射到Pod(容器)内部的/data目录；

参考：
https://help.aliyun.com/document_detail/86784.html

静态卷

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-disk
  labels:
    failure-domain.beta.kubernetes.io/zone: cn-shenzhen-a
    failure-domain.beta.kubernetes.io/region: cn-shenzhen
    pvname: pv-disk
spec:
  capacity:
    storage: 20Gi
  accessModes:
    - ReadWriteOnce
  flexVolume:
    driver: "alicloud/disk"
    fsType: "ext4"
    options:
      volumeId: "d-wz9bkj7jhbb4jo77n1tz"
---
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pvc-disk
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
  selector:
    matchLabels:
      pvname: pv-disk
```

动态卷

```
kind: StorageClass
apiVersion: storage.k8s.io/v1beta1
metadata:
  name: alicloud-disk-ssd-shenzhen-a
provisioner: alicloud/disk
reclaimPolicy: Retain
parameters:
  type: cloud_ssd
  regionid: cn-shenzhen
  zoneid: cn-shenzhen-a
  encrypted: "true"
---
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: disk-ssd1
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: alicloud-disk-ssd-shenzhen-a
  resources:
    requests:
      storage: 20Gi
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  ports:
  - port: 80
    name: web
  selector:
    app: nginx
---
apiVersion: apps/v1beta2
kind: StatefulSet
metadata:
  name: web
spec:
  selector:
    matchLabels:
      app: nginx
  serviceName: "nginx"
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx
        volumeMounts:
        - name: disk-ssd
          mountPath: /data
      volumes:
        - name: "disk-ssd"
          persistentVolumeClaim:
            claimName: pvc-disk
```

```
# kubectl get pod
NAME    READY   STATUS    RESTARTS   AGE
web-0   1/1     Running   0          5m1s

# kubectl exec -ti web-0 mount | grep data
/dev/vdb on /data type ext4 (rw,relatime,data=ordered)

#  mount | grep disk
/dev/vdb on /var/lib/kubelet/plugins/kubernetes.io/flexvolume/alicloud/disk/mounts/pv-disk type ext4 (rw,relatime,data=ordered)
/dev/vdb on /var/lib/kubelet/pods/6b629120-da15-11e9-b200-00163e007d53/volumes/alicloud~disk/pv-disk type ext4 (rw,relatime,data=ordered)
```


阿里云 CLOUD NATIVE COMPUTING FOUNDATION

1

Kubernetes
存储体系架构

2

Flexvolume
介绍及使用

3

CSI
介绍及使用

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

CSI 介绍

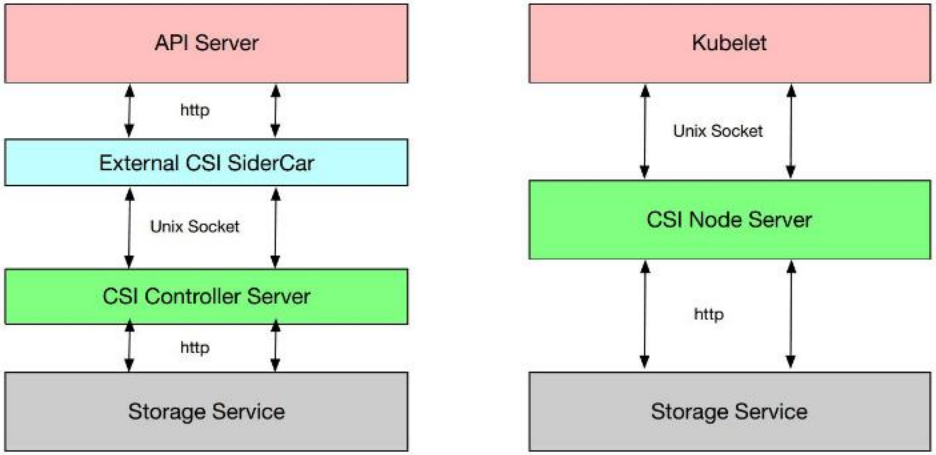
同Flexvolume一样，为第三方存储实现数据卷挂载实现的抽象接口；

有了Flexvolume，为何还要CSI?

- 提供容器存储统一接口，实现编排系统与存储类型解耦，在多平台运行（K8S, Mesos, Swarm）；
- 容器化部署，减少环境依赖，增强安全性，丰富插件功能；

CSI包含两个部分：

- CSI Controller Server：实现控制端的创建、删除、挂载、卸载功能；
- CSI Node Server：实现节点上的mount、Unmount功能；



通用管控接口	
GetPluginInfo	获取Driver身份信息，如Name;
GetPluginCapabilities	获取提供的能力，是否支持ControllerService？是否支持在线扩容?
Probe	插件健康检查接口；
节点管控接口	
NodeStageVolume	在节点上初始化存储卷(格式化)，并执行挂载到Global目录；
NodeUnstageVolume	Unmount存储卷在节点上的Global目录；
NodePublishVolume	在节点上将存储卷的Global目录挂载到Pod挂载目录；
NodeUnpublishVolume	Unmount存储卷在节点上的Pod挂载目录；
NodeGetVolumeStats	获取节点上Volume挂载文件系统统计信息(总空间、可用空间等)；
NodeExpandVolume	扩容存储卷，在节点上扩容文件系统等；
NodeGetCapabilities	获取Node Plugin提供的能力列表；
NodeGetInfo	获取节点信息：节点ID、最大挂载数、拓扑信息；
中心管控接口	
CreateVolume	创建存储卷，包括远端存储介质，以及PV对象；
DeleteVolume	删除存储卷，包括远端存储介质，以及PV对象；
ControllerPublishVolume	挂载存储卷：将存储介质挂载到目标节点；
ControllerUnpublishVolume	卸载存储卷：将存储介质从目标节点卸载；
ValidateVolumeCapabilities	验证Controller信息，AccessMode、VolumeType等；
ListVolumes	获取Volume列表；
GetCapacity	获取存储池可创建卷的size能力，可用参数、Topo作为限定条件；
ControllerGetCapabilities	获取Controller Plugin提供的能力列表；
CreateSnapshot	创建存储卷快照，并创建快照对象(VolumeSnapshot)；
DeleteSnapshot	删除存储卷快照，并删除快照对象(VolumeSnapshot)；
ListSnapshots	获取VolumeSnapshot对象列表；
ControllerExpandVolume	执行存储卷扩容，并不包含文件系统扩容；

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

CSI 对象

VolumeAttachments：

- 描述一个Volume卷挂载、卸载相关的信息。包含：卷的名字、挂载目的节点、使用的插件、当前状态等信息。代表一个挂载操作期望；
- AD Controller 在执行挂载一个CSI PV的时候，会调用csi-attacher(In-Tree)创建一个VolumeAttachment；
- External-attacher 通过watch VolumeAttachment，发现有需要挂载的数据卷时，调用csi-plugin的ControllerPublishVolume方法，执行attach操作。
- VolumeAttachment是由AD Controller调用csi-attacher删除。

CSIDriver：

- 描述集群中部署的CSI Plugin列表，需要管理员根据插件类型创建；
- 定义Kubernetes调用CSI Plugin的行为：
  - AttachRequired: 定义Plugin是否支持Attach功能，如果定义为false，Kubernetes不会调用Attach接口；
  - podInfoOnMount: 定义Kubernetes调用Mount接口时，是否带上Pod信息；
- 已经Beta，Kubernetes API已经默认支持，可以直接创建使用；
- 1.13之前版本需要手动打开Feature Gate，并创建CSI Driver；

CSINode：

- 定义集群中CSI节点信息，node-driver-registrar启动时创建；
- CSINode的作用：
  - 每一个新的CSI Plugin注册后，都会在CSINode里添加一个CSINode信息；
  - Controller Manager、Scheduler依赖CSINode信息来确定一个节点的CSI驱动是否可用；
  - Controller Manager实现Topoloy-Aware Provsion的时候，通过CSINode信息获取拓扑信息；
- 已经Beta，Kubernetes API已经默认支持，可以直接创建；
- 1.13之前版本需要手动打开Feature Gate，并创建CSINodeInfo；

```
# kubectl get volumeattachment csi-***** -oyaml
apiVersion: storage.k8s.io/v1
kind: VolumeAttachment
metadata:
  name: csi-*****
spec:
  attacher: ossplugin.csi.alibabacloud.com
  nodeName: cn-zhangjiakou.192.168.1.53
  source:
    persistentVolumeName: oss-csi-pv
status:
  attached: false
```

```
apiVersion: storage.k8s.io/v1beta1
kind: CSIDriver
metadata:
  name: mycsidriver.example.com
spec:
  attachRequired: true
  podInfoOnMount: true
```

```
# kubectl get csidriver
NAME                          CREATED AT
diskplugin.csi.alibabacloud.com   2019-09-05T13:16:59Z
nasplugin.csi.alibabacloud.com    2019-09-05T13:16:59Z
ossplugin.csi.alibabacloud.com    2019-09-05T13:16:59Z
```

```
# kubectl get csinode cn-zhangjiakou.192.168.1.49 -oyaml
apiVersion: storage.k8s.io/v1beta1
kind: CSINode
metadata:
  name: cn-zhangjiakou.192.168.1.49
spec:
  drivers:
  - name: diskplugin.csi.alibabacloud.com
    nodeID: i-8vb11m0jv29opj6mn3fn
    topologyKeys:
    - topology.diskplugin.csi.alibabacloud.com/zone
  - name: nasplugin.csi.alibabacloud.com
    nodeID: i-8vb11m0jv29opj6mn3fn
    topologyKeys: null
```

```
# kubectl get csinodes
NAME                          CREATED AT
cn-zhangjiakou.192.168.1.49   2019-09-05T13:17:15Z
cn-zhangjiakou.192.168.1.50   2019-09-05T13:17:16Z
cn-zhangjiakou.192.168.1.51   2019-09-05T13:17:16Z
cn-zhangjiakou.192.168.1.52   2019-09-05T13:17:16Z
cn-zhangjiakou.192.168.1.53   2019-09-05T13:17:17Z
```


阿里云 CLOUD NATIVE COMPUTING FOUNDATION

CSI 组件 - Node-Driver-Registrar

负责与Kubelet配合实现CSI 插件的注册功能：

1. Node-driver-registrar 向CSI Plugin调用GetPluginInfo接口，获得Plugin DriverName；

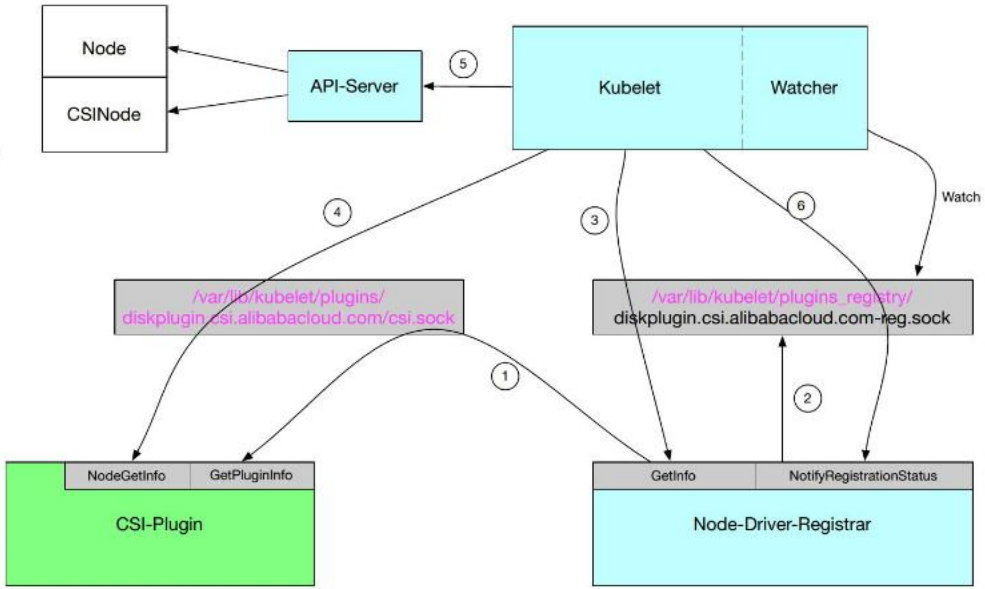
2. 启动监听服务，监听：/var/lib/kubelet/plugins_registry/diskplugin.csi.alibabacloud.com-reg.sock 地址；监听2个接口：
- GetInfo：提供Plugin的相关信息，类型、名字、实际插件socket地址、版本；
- NotifyRegistrationStatus：接收kubelet的Plugin注册结果，注册失败时插件会停止；

3. kubelet向Node-driver-registrar调用GetInfo，获取plugin的信息（名字、实际插件的socket地址）(由Watcher发现socket文件后触发调用操作)；

4. kubelet向真实csi插件发送NodeGetInfo请求，获取：NodeID、节点拓扑等插件信息；将CSI插件加入Plugin列表；

5. 调用Kubernetes API，更新Node信息：
- 更新节点：Annotations：添加csi.volume.kubernetes.io/nodeid信息；
- 更新节点：Labels：添加拓扑信息，topology.diskplugin.csi.alibabacloud.com/zone=*
- 更新节点：status.allocatable 和 status.capacity 字段，添加MaxAttachLimit信息；
- 创建(更新)CSINode：添加节点对应的插件信息（nodeId，driverName，拓扑信息等）；

6. 调用Node-driver-registrar的NotifyRegistionStatus接口，通知Plugin已经注册成功(或失败)；



阿里云 CLOUD NATIVE COMPUTING FOUNDATION

CSI 近期Features

ExpandCSIVolumes：

- 实现数据卷的扩容，包括文件系统扩容；
- 需要部署Sidercar：csi-resizer；
- 实现：ControllerExpandVolume, NodeExpandVolume；
- 1.14 Alpha，1.16 Beta；

VolumeSnapshotDataSource：

- 实现数据卷的快照功能；
- 需要部署SiderCar：csi-snapshotter；
- 实现：CreateSnapShot, DeleteSnapshot接口；
- 1.12 Alpha；

VolumePVCDataSource：

- 创建PVC的时候，以其它Volume作为数据源进行拷贝创建；
- 需要在PVC中定义dataSource参数；
- 功能启动：--feature-gates=VolumePVCDataSource=true
- 1.15 Alpha, 1.16 Beta;

CSIInlineVolume:

- 运行CSI Volume直接在Pod中定义，而不需通过PVC、PV的方式定义；
- 功能开启：--feature-gates=CSIInlineVolume=true
- 1.15 Alpha, 1.16 Beta;

项目示例：
https://github.com/kubernetes-sigs/alibaba-cloud-csi-driver

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

谢谢观看

THANK YOU